KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

Government Scheme Navigator

Project Documentation

Project ID: PRJ084

Team ID: T084

Team ID: SPIRIT

Team GitHub Repository: https://github.com/akanksha-mandala/AccessgovAI
Live Link : https://accessgovai-1.onrender.com

Student Name	Registration Number	Individual GitHub Link
MANDALA AKANKSHA	99230040366	https://github.com/akanksha-mandala
KUNAPAREDDY BHARGAV TEJA	99230040980	https://github.com/kunapareddybhargav93-tech
PALEPU MOULYA	99230040391	https://github.com/Palepumoulya
SHAIK ABDUL SATTAR	99230040948	https://github.com/Abdul-123-art
MIDDOLA YESASWINI	99230040375	https://github.com/Yesaswini05

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1.Introduction……...Section 1
2. Problem Statement, Objectives & Scope…… ……....Section 2
3. System Architecture and RAG Pipeline .....Section 3
4. Implementation Details and Technologies Used …..…….Section 4
5. Results, Testing and Performance Evaluation .....Section 5
6.Main Features……Section 6
7. Conclusion and Future Scope .....Section 7
8. References .....Section 8

1.Introduction

Government schemes and public services are intended to provide support to citizens in areas such as education, financial assistance, certificates and social welfare. However, finding the correct scheme and understanding its requirements can be difficult. Information is often spread across different departments and websites, and citizens may not always understand official terminology, eligibility conditions or the documents they need to prepare.

Government Scheme Navigator is a web-based application developed to make this process easier. It provides a common platform where citizens can explore government services, understand eligibility requirements, check required documents and receive guidance through an AI-powered conversational assistant.

The application focuses on practical citizen needs, particularly Scholarship and Income Certificate services. Instead of making users search through lengthy descriptions, the system presents service information in a more understandable format. Citizens can ask questions through the chatbot, use voice input, change the preferred language and upload documents for an automated readability and readiness assessment.

The project combines a React-based frontend with a Python FastAPI backend. Gemini 2.5 Flash is integrated for conversational assistance, while Retrieval-Augmented Generation (RAG) is used to provide relevant service information as context for AI-generated responses. ChromaDB stores and retrieves vector representations of knowledge, and Sentence Transformers converts text into embeddings.

The application also includes document processing using PaddleOCR, rule-based eligibility evaluation, PostgreSQL database management and accessibility features. The main aim is to reduce the effort involved in understanding government services and help citizens prepare the information and documents required before approaching the relevant authority.

2. Problem Statement, Objectives & Scope

2.1 Problem Statement

Citizens often face difficulties when trying to identify government schemes that match their requirements. Even after finding a scheme, they may not clearly understand its eligibility conditions, required documents, application procedure, fees or processing time.

The problem becomes more challenging for people who are unfamiliar with government terminology, have limited digital experience or prefer communicating in regional languages. Manually checking multiple sources can also take time and may lead to confusion when information is incomplete or outdated.

There is a need for a simple platform that brings relevant service information together, explains requirements in understandable language and helps citizens prepare their documents. Government Scheme Navigator addresses this need through service discovery, rule-based eligibility assistance, conversational AI, multilingual interaction and document-readiness assessment.

2.2 Objectives

The main objectives of Government Scheme Navigator are:

1. To develop a web-based platform that helps citizens discover relevant government schemes and public services through a common interface.
2. To provide information about service eligibility, required documents, departments, fees and estimated processing time.
3. To implement a conversational assistant using Gemini 2.5 Flash so that citizens can ask questions and receive explanations in a more natural way.

4. To use Retrieval-Augmented Generation (RAG) to retrieve relevant government-service information and provide contextual information to the language model before response generation.
5. To implement a rule-based eligibility engine that compares available citizen details with configured service conditions and identifies missing information.
6. To provide document upload and analysis using PaddleOCR, document classification, field extraction and validation.
7. To calculate a document-readiness score so that citizens can identify incomplete fields and documents that may need to be uploaded again.
8. To support language selection and voice interaction to make the platform easier to use for citizens with different language preferences.
9. To maintain user, service, document and application-related information using a PostgreSQL database.

2.3 Scope of the Project

Government Scheme Navigator is designed as a citizen-assistance platform rather than a replacement for official government portals.

The current implementation concentrates on Scholarship and Income Certificate-related workflows. It also includes a service catalogue containing other government-service records, allowing the platform to present information such as department details, eligibility descriptions, required documents and processing information.

The scope includes service discovery, eligibility assistance, document requirement guidance, conversational question answering, multilingual response support, voice input, document upload and readiness assessment. An administrative interface is also included for viewing service-related analytics and platform information.

The application helps citizens understand and prepare for a service. It does not independently approve certificates, certify document authenticity or submit applications directly to

government departments. Final verification and approval remain with the concerned government authorities.

3. System Architecture and RAG Pipeline

3.1 Overall System Architecture

Government Scheme Navigator follows a frontend-backend architecture in which the user interface communicates with a Python-based backend through REST APIs.

The frontend is developed using React and TypeScript. It provides the citizen dashboard, service catalogue, eligibility checker, document upload interface and conversational assistant. User actions are sent to the backend, where the relevant API and service modules process the request.The backend is developed using FastAPI. It handles authentication, service discovery, eligibility evaluation, document processing, conversation management and database operations.

PostgreSQL stores application information, including user details, government services, service requirements, uploaded-document records and other platform data.The AI assistant uses a conversation agent to coordinate user requests. Depending on the type of question, the system can invoke service discovery, eligibility, document-requirement or explanation tools. Relevant knowledge can also be retrieved through the RAG module and supplied to Gemini 2.5 Flash as context.

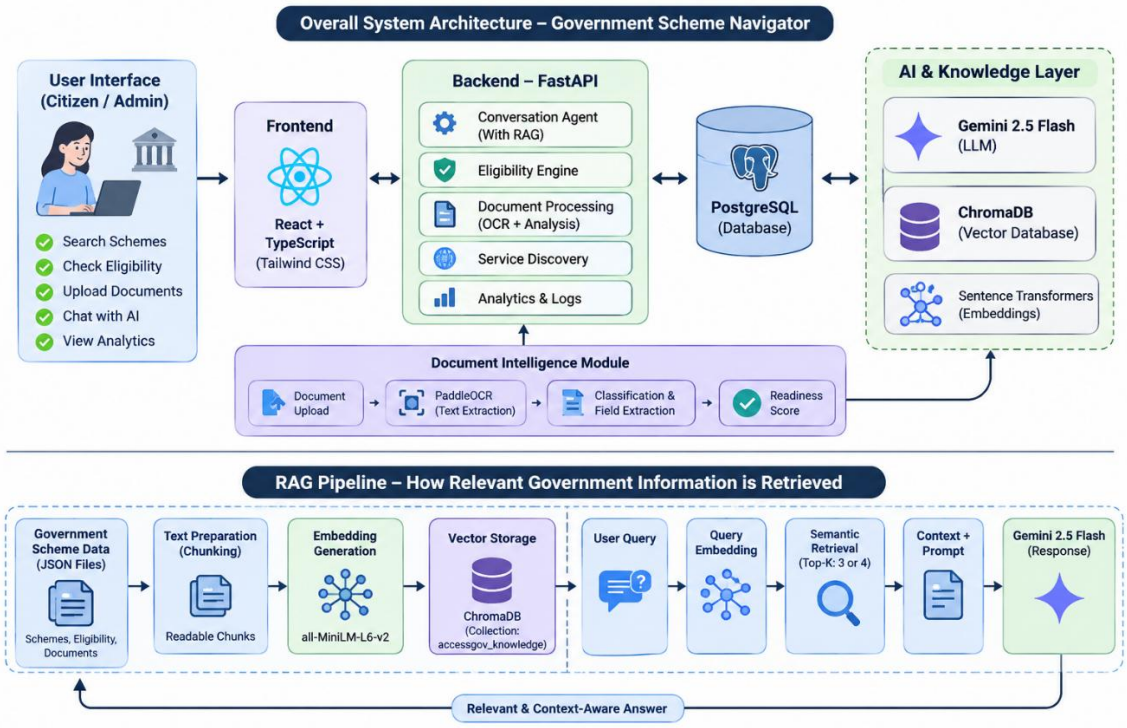


Figure 1: Overall System Architecture of Government Scheme Navigator

3.2 RAG Pipeline

Retrieval-Augmented Generation is used to provide the language model with relevant information from an external knowledge collection before it generates an answer.

In this project, the RAG pipeline is designed around government-service information stored in JSON datasets. These datasets contain service descriptions, departments, eligibility details, required documents and related information.

The RAG implementation consists of the following stages:

1. **Knowledge Input:** Government-service information is maintained in JSON datasets, including service catalogue records and specific scholarship and income-certificate rules.
2. **Text Preparation:** The ingestion module reads the JSON records and prepares readable text chunks containing service names, departments, descriptions, eligibility details and required documents.
3. **Embedding Generation:** The Sentence Transformers model all-MiniLM-L6-v2 converts the text chunks into numerical vector representations.
4. **Vector Storage:** The generated embeddings, text chunks and associated metadata are stored in a persistent ChromaDB collection named accessgov_knowledge.
5. **Query Embedding:** When a citizen asks a question, the same embedding model converts the query into a vector.
6. **Semantic Retrieval:** ChromaDB compares the query vector with stored vectors and retrieves the most relevant knowledge chunks using similarity search.
7. **Context Preparation:** The retrieved information is formatted and included in the prompt sent to Gemini.
8. **Response Generation:** Gemini 2.5 Flash uses the supplied context, conversation history and selected language to generate a citizen-friendly response.

The retriever supports Top-K retrieval. The default value is 4, while the conversation agent explicitly uses 3 or 4 in different retrieval operations.

The service discovery module also combines SQL keyword search with ChromaDB semantic search. This allows the system to consider both direct keyword matches and similarity in meaning when ranking government services.

RAG helps reduce responses based entirely on the model's general pretrained knowledge. However, the quality of the answer still depends on the correctness and completeness of the stored information and the relevance of the retrieved context.

4. Implementation Details and Technologies Used

4.1 Technology Stack

The following technologies are used to develop InterviewGen AI:

Component	Technology Used	Purpose
Frontend	React, TypeScript, Vite	Building the citizen and administrative interfaces
UI Styling	Tailwind CSS	Layout, styling and responsive interface design
Backend	Python, FastAPI	API development and application logic
Database	PostgreSQL	Storing user, service, document and application information
ORM	SQLAlchemy	Database interaction through Python models
AI Model	Gemini 2.5 Flash	Conversational responses, explanations and language processing
AI Integration	Google ADK components and Gemini API	Agent configuration and model interaction

Vector Database	ChromaDB	Storing and retrieving semantic knowledge embeddings
Embedding Model	Sentence Transformers – all-MiniLM-L6-v2	Converting text and queries into vector embeddings
OCR	PaddleOCR	Extracting text from uploaded document images
PDF Processing	PyPDF, pdf2image	Reading searchable PDFs and rendering scanned PDF pages
Document Processing	OpenCV, Pillow	Image and document handling
Authentication	JWT	Managing authenticated access
Voice Processing	Web Speech API	Speech recognition and text-to-speech interaction
Frontend Navigation	React Router DOM	Navigating between application pages
Development Environment	Visual Studio Code	Coding, testing and debugging

4.2 RAG Implementation

The RAG implementation is divided into separate modules so that embedding generation, data ingestion, retrieval and context formatting can be handled independently.

Main RAG modules:

- ingest.py – Reads JSON knowledge datasets, prepares text chunks and indexes them in ChromaDB.
- embeddings.py – Generates vector embeddings using all-MiniLM-L6-v2.
- retriever.py – Performs semantic similarity search and retrieves relevant knowledge chunks.

- formatter.py – Formats retrieved information for use in the conversation prompt.
- prompts.py – Contains instructions for generating citizen-friendly and context-based responses.

ChromaDB uses a persistent collection so that indexed information can be stored locally. The ingestion module provides the functionality to load the prepared datasets and populate this collection.

The ingestion process must be executed to populate the vector collection; the application source does not show an automatic ingestion call during startup.

4.3 Database Design

PostgreSQL is used as the main relational database. SQLAlchemy provides the connection between the Python backend and database tables.

The main data entities include:

Data Entity	Information Stored	Purpose
User Information	User details, login information and roles	Registration, authentication and access control
Government Services	Service names, codes, departments and descriptions	Maintaining the service catalogue
Service Requirements	Required document names and related details	Displaying service-specific document requirements
Uploaded Documents	File details, document type and processing status	Maintaining citizen document records
Applications	Application-related information and status fields	Organising citizen application records

Conversation Logs	Conversation-related information	Maintaining assistant interaction records
Analytics Data	Platform and service-related metrics	Supporting administrative dashboards

JWT-based authentication is used to protect authenticated operations. Role-based access control is included to distinguish citizen and administrative access.

4.4 AI Model Integration

The application integrates Gemini 2.5 Flash through the Gemini API and Google ADK-related agent components.

The conversation agent acts as the coordinator between the citizen's query and the backend services. It maintains conversation context, identifies the user's intent and delegates the request to the appropriate tool.

The intent router uses a two-stage approach:

1. **Keyword and rule-based classification:** It checks the user's message against predefined intent keywords.
2. **Gemini fallback classification:** If the rule-based classification does not reach the required confidence threshold, Gemini is used as a fallback to classify the request.

The supported intent categories include service discovery, eligibility checking, document requirements, simple explanations, application guidance and greetings.

The model is pretrained. The project does not train or fine-tune Gemini. Instead, it uses prompts, retrieved information and conversation context to guide the responses.

4.5 Service Discovery and Eligibility Evaluation

The service catalogue module combines SQL keyword matching with ChromaDB semantic retrieval. The results are merged, ranked and deduplicated to identify services relevant to the citizen's query.

The eligibility engine follows a rule-based approach rather than asking Gemini to make the eligibility decision. It evaluates available profile information against configured conditions, such as annual income, age, disability percentage, student status and required documents.

The output includes an eligibility status, eligibility score, missing information, missing documents and suggested next steps.

This separation is important because eligibility calculations are handled through defined application logic, while Gemini is used for conversational explanation.

4.6 Document Intelligence and OCR

The document intelligence module helps citizens understand whether an uploaded document is readable and whether the extracted information appears complete.

The processing flow is:

1. The citizen uploads a supported document in PDF, JPG, JPEG or PNG format.
2. The backend validates the file and stores it.
3. PaddleOCR extracts text from supported document images.
4. For searchable PDFs, PyPDF is used to extract available text.
5. For scanned PDFs without extractable text, pdf2image can convert up to the first three pages into images for OCR processing.
6. The document classifier identifies the likely document type using keywords, regular expressions and fuzzy matching.
7. The field extractor identifies supported information from the extracted text.

8. Sensitive identifiers such as Aadhaar, PAN and bank account numbers are masked in the extracted output.
9. The validator checks the extracted fields and identifies missing or invalid information.
10. The readiness engine calculates a document-readiness score and generates suggested next steps.

The readiness score is calculated using three components:

- Completeness of extracted fields: maximum 50 points.
- OCR text confidence: maximum 30 points.
- Field validity: 20 points if valid, otherwise 5 points.

The final score is limited to a range of 0–100.

The score indicates document readiness based on the application's checks. It does not establish document authenticity or replace official verification by government officers.

4.7 Multilingual and Voice Interaction

The platform provides language selection for English, Tamil, Telugu, Hindi and Kannada.

The frontend uses a shared accessibility context to maintain the selected language. Translation dictionaries are used for supported interface text, while the backend translation service uses Gemini for translating generated responses when required.

Voice interaction is implemented using the browser's Web Speech API.

The speech-recognition locale mappings include:

Language	Recognition Locale
English	en-IN
Tamil	ta-IN

Telugu	te-IN
Hindi	hi-IN
Kannada	kn-IN

The recognised speech is converted into text and sent to the conversation API. The backend processes the request, and the frontend uses SpeechSynthesis to speak the response.

Page navigation is handled separately using React Router DOM and useNavigate(). The voice component matches supported navigation keywords to application routes. The current route-matching logic is primarily English-based, so full regional-language voice navigation is an area for further improvement.

4.8 Technology Selection

React and TypeScript were selected to build a structured and interactive user interface. FastAPI provides a Python-based backend suitable for connecting application services with AI and document-processing modules.

PostgreSQL was selected for relational data management, while SQLAlchemy simplifies database operations. Gemini 2.5 Flash provides conversational response generation, and Google ADK-related components help organise the agent workflow.

ChromaDB was selected because it provides persistent local vector storage and semantic retrieval without requiring a mandatory paid cloud database. Sentence Transformers provides a locally executable embedding model, making the retrieval component suitable for a cost-conscious prototype.

PaddleOCR was included to extract text from uploaded documents, while the rule-based readiness engine provides a predictable scoring method that does not depend on an LLM-generated score.

5.Overview of the System or System architecture

5.1 Results

Government Scheme Navigator has been developed as a web-based citizen-assistance application that brings together government-service discovery, eligibility assistance, document guidance and conversational interaction.

The citizen interface provides access to the service catalogue, eligibility checker, document upload page and conversational assistant. The backend exposes separate API modules for authentication, service discovery, eligibility, conversation and document processing.

The service catalogue supports keyword and semantic matching. The eligibility engine returns structured results based on the information supplied by the citizen. The document intelligence pipeline extracts text, identifies supported document types, checks fields and calculates a readiness score.

The conversational assistant combines intent classification, backend tools, retrieved context and Gemini response generation. Language selection and voice interaction provide additional ways for citizens to interact with the platform.

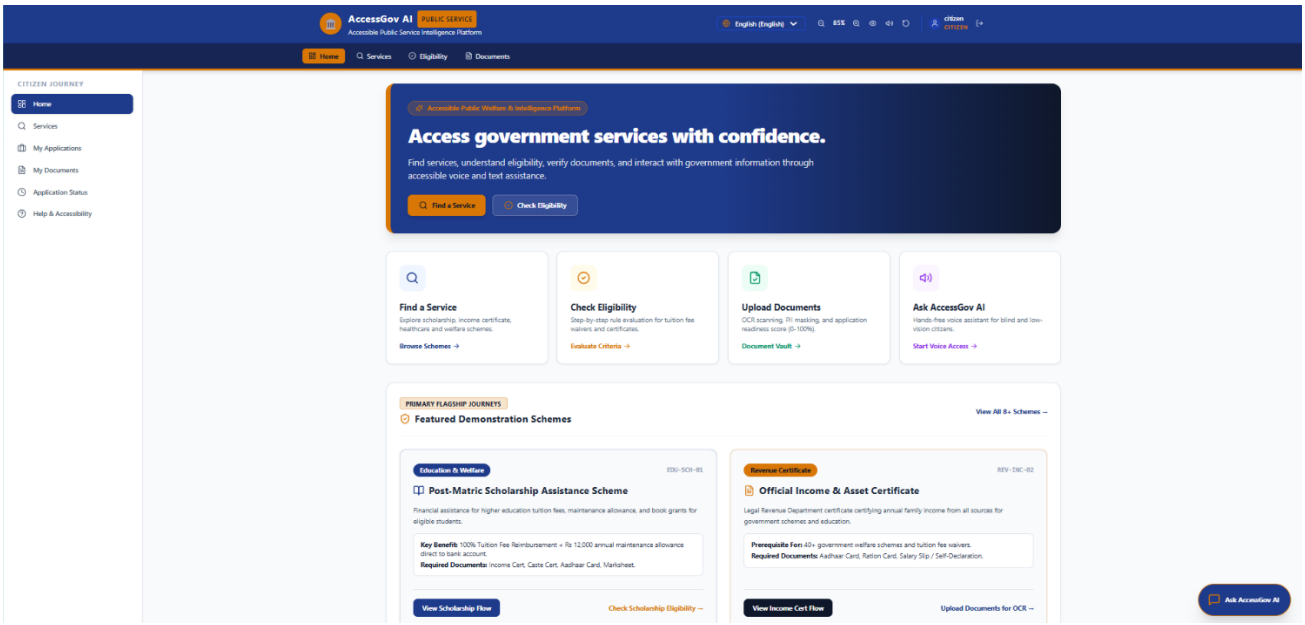
The administrative interface includes dashboards for service-related analytics, document analytics, conversation analytics and accessibility-related information.

The system demonstrates how AI-based assistance, retrieval, rule-based evaluation and document processing can be combined in a single citizen-facing application.

5.2 Testing

The backend contains test modules covering conversation handling, document classification, document intelligence, eligibility evaluation, OCR, RAG, readiness calculation and service operations.

The following test areas are considered in the project:



5.2 Testing

User Authentication	Registration and login requests	Valid users should receive authenticated access
Service Discovery	Searching using service names and related terms	Relevant service records should be returned
Eligibility Evaluation	Submitting citizen profile information	Eligibility status, missing conditions and next steps should be returned
Document Requirements	Requesting requirements for a selected service	Required documents should be displayed
Conversation Agent	Sending questions through the chat interface	The request should be classified and processed
RAG Retrieval	Searching the knowledge collection	Relevant stored text chunks should be retrieved

OCR Processing	Uploading supported document images and PDFs	Readable text should be extracted where possible
Document Classification	Processing extracted document text	A supported document type should be identified
Readiness Calculation	Submitting extracted fields and validation results	A score and improvement suggestions should be generated
Language and Voice	Changing language and using speech input	Supported language and speech functions should respond as configured

he actual pass/fail status should be recorded after running the corresponding tests in the target environment.

5.3 Performance Evaluation

The system is evaluated based on the functionality and behaviour of its individual modules.

Service discovery is designed to use both keyword matching and semantic similarity. The RAG module provides relevant context to the conversational assistant, while the rule-based eligibility engine produces structured results from the available citizen information.

Document processing is evaluated through OCR extraction, classification, field validation and readiness scoring. The scoring method is deterministic, allowing the same extracted fields and validation result to produce the same readiness score.

The conversational assistant depends on the availability of the Gemini API for AI-generated responses. Local OCR and vector operations can run without a cloud AI request once the required dependencies, models and indexed data are available.

No fixed accuracy percentage or response-time benchmark is claimed because a separate quantitative evaluation has not been established.

5.4 Overall Outcome

The project demonstrates a practical approach to making government-service information easier to understand and access.

Instead of treating service discovery, eligibility checking, document preparation and citizen interaction as separate activities, the platform brings them together through a common interface.

The combination of a service catalogue, rule-based evaluation, RAG retrieval, conversational AI, voice interaction and document readiness assessment provides a foundation for a more accessible government-service assistance system.

6.Main Features

Government Scheme Navigator includes the following major features:

1. **Citizen Registration and Authentication:** Allows citizens to create accounts and access protected application functions using authentication mechanisms.
2. **Government Service Catalogue:** Displays available service information, including service descriptions, departments, eligibility details, fees and processing information.
3. **Service Discovery:** Helps citizens find relevant services using search queries, keyword matching and semantic retrieval.
4. **Eligibility Checker:** Evaluates the citizen's available profile information against configured eligibility conditions and identifies missing details.
5. **Required Document Guidance:** Displays the documents associated with a selected government service and helps citizens understand what they need to prepare.
6. **AI-Powered Conversational Assistant:** Allows citizens to ask questions in natural language and receive contextual explanations through Gemini 2.5 Flash.
7. **RAG-Based Knowledge Retrieval:** Retrieves relevant government-service information from the ChromaDB knowledge collection and supplies it as context to the language model.

8. **Multilingual Interaction:** Provides language selection and translation support for English, Tamil, Telugu, Hindi and Kannada.

9. **Voice-Based Interaction:** Uses browser speech recognition to convert spoken input into text and text-to-speech functionality to read responses aloud.

10. **Document Upload and OCR:** Allows authenticated citizens to upload supported documents for text extraction and document analysis.

11. **Document Classification and Field Extraction:** Identifies supported document types and extracts relevant fields while masking selected sensitive identifiers.

12. **Document Readiness Score:** Evaluates field completeness, OCR confidence and field validity to help citizens identify document-quality issues.

13. **Application and Document Records:** Maintains relevant citizen records and uploaded-document information in the database.

14. **Accessibility Features:** Includes accessibility controls and language-related interface support to improve usability for citizens with different needs.

15. **Administrative Analytics:** Provides administrative views for document, conversation, accessibility and district-related analytics.

7. Conclusion and Future Scope

Government Scheme Navigator was developed to address a practical problem faced by citizens: understanding which government services are relevant to them and what they need to prepare before applying.

The platform brings service discovery, eligibility assistance, document guidance and conversational support into one web application. It uses a rule-based engine for structured eligibility evaluation, RAG for retrieving relevant service information, and Gemini 2.5 Flash for generating natural-language responses.

The document intelligence module adds another useful layer by extracting information from uploaded documents and providing a readiness score based on readability, completeness and validation.

Language selection, voice interaction and accessibility controls are included to make the platform easier to use.

The project demonstrates the integration of web development, database management, AI-based conversation, semantic retrieval and OCR in a public-service assistance application. It is a prototype intended to help citizens understand and prepare for government services; official application processing and approval remain the responsibility of government authorities.

7.2 Future Scope

The project can be improved in the following areas:

1. **Expanded Government-Service Coverage:** Add more central and state government schemes, certificates and welfare services with carefully maintained eligibility and document information.

2. **Automatic Knowledge-Base Updates:** Integrate verified government information sources so that changes in eligibility rules, fees and document requirements can be reflected more regularly.

3. **Improved Multilingual Voice Navigation:** Extend voice-command interpretation so that citizens can navigate application pages using natural commands in all supported regional languages.

4. **Improved RAG Retrieval:** Introduce better document chunking, retrieval evaluation, relevance thresholds and source attribution to improve the quality of retrieved information.

5. **Advanced Document Intelligence:** Improve OCR handling for handwritten text, low-quality images, regional-language documents and multi-page scanned PDFs.

6. **Official Portal Integration:** Explore secure integration with authorised government portals for application submission and status tracking, subject to official API availability and permission.

7. **Improved Accessibility:** Add more support for screen readers, keyboard-only navigation, simplified workflows and additional accessibility requirements.

8. **Stronger Information Verification:** Add source links, update dates and review mechanisms so citizens can identify where service information originated and whether it has been recently checked.

9. **Enhanced Analytics:** Provide better service-usage insights and administrative reports while protecting citizen privacy.

8.References

1. P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. https://arxiv.org/abs/2005.11401
2. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP-IJCNLP*, 2019. https://arxiv.org/abs/1908.10084
3. Chroma, "Chroma Documentation – Vector Database and Semantic Search." https://docs.trychroma.com/
4. Google AI for Developers, "Gemini API Documentation." https://ai.google.dev/gemini-api/docs
5. Google, "Agent Development Kit Documentation." https://google.github.io/adk-docs/
6. FastAPI, "FastAPI Documentation." https://fastapi.tiangolo.com/
7. PostgreSQL Global Development Group, "PostgreSQL Documentation." https://www.postgresql.org/docs/
8. PaddlePaddle, "PaddleOCR Documentation." https://www.paddleocr.ai/
9. Hugging Face, "Sentence Transformers Documentation." https://www.sbert.net/
10. MDN Web Docs, "Web Speech API." https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API